

```
// =====
// FANTASY PvP ENGINE v2.4.0 – motor de pontuação isolado
// Não depende de nenhum jogo específico: recebe (player, match, tactic)
// e devolve a pontuação detalhada. Mesmo motor pra todas as salas.
// =====

// Pesos de pontuação. BASE = rebalanceado (gol/assist valem mais, passe/desarme
// BASE_V1 = pesos antigos, usados SÓ pelos jogos já apurados (match.tacticRules=
// pra não recalcular pontuações que já valeram.
const BASE = { goal:4.6, assist:3.6, sot:2.2, dribble:.85, prgp:.06, pib:0, tib:0
  tklint:.36, block:.48, recovery:.05, aerial:.16, clearance:.03,
  save:1.4, penSave:6, opa:1.35, crossStop:.8, accCross:.2, inaccCross:-.08,
  wasFouled:.08, longBall:.12, prgCarry:.10, penaltyWon:2.5,
  psConv:1.5, psMiss:-4.0, psSave:3.0,
  yellow:-2, redH1:-7, redH2:-5, errGoal:-5, errShot:-2, penCom:-4, dribbledPast:
// pesos ANTIGOS – CONGELADOS (jogos já apurados / tacticRules v1). NÃO herdam da
const BASE_V1 = { goal:2, assist:1.5, sot:0.6, dribble:0.35, prgp:0.12, pib:0.35,
const CAPS = { MATCH:28, FLOOR:-9, CLUTCH:8, TACT:13 };
// multiplicador de equilíbrio por posição (só nos pontos POSITIVOS) – normaliza
// POS_MULT_LEGACY = calibração antiga (amostra de 5 jogos). CONGELADA: usada pel
// apurados (match.posMultLegacy===true) pra não alterar pontuações que já vale
// POS_MULT = calibração "Forte" rebalanceada com 60 jogos apurados (~1891 jogado
// encosta as médias por posição (GK/DEF subem, MID/ATT descem) deixando todas
// Vale pros jogos NOVOS. Jogos v1 continuam usando 1.0 (ficam intactos).
const POS_MULT_LEGACY = { GK:1.02, DEF:1.0, MID:1.04, ATT:1.09 };
const POS_MULT = { GK:1.25, DEF:1.14, MID:0.95, ATT:0.93 };
const TIER_EMO = {1:0,2:.4,3:.9,4:1.6};
const r1 = x => Math.round(x*10)/10;
const tierXG = v => v>0.5?{b:0,t:1} : v>=0.2?{b:1.2,t:2} : v>=0.08?{b:2.6,t:3} :
const tierSV = v => v<0.1?{b:0,t:1} : v<=0.3?{b:1.2,t:2} : v<=0.6?{b:2.6,t:3} : {

// ---- TÁTICAS v3.0 – baseadas na SUA escalação ----
// A condição olha squadSum = soma das stats dos SEUS jogadores que TERMINARAM
// a partida em campo (titular não-substituído OU reserva que entrou e terminou).
// Se ativar: buff ×1.18 nas ações premiadas, nerf ×0.90 nas penalizadas.
// buffs/nerfs aplicam a CADA jogador do seu time individualmente.
// _____
// TÁTICAS v3 – SKILL, não sorte.
// A condição olha a COMPOSIÇÃO do time que você montou (posições + preços
// dos titulares que terminaram em campo), não o que aconteceu na partida.
// Você controla 100% se a tática ativa, montando o time do jeito certo.
// Efeito liga/desliga: se a composição bate, todos ganham o buff (×1.18).
// SEM nerfs – errar a tática só significa não ganhar o bônus, sem punição.
// `comp` traz: nGK,nDEF,nMID,nATT (titulares por posição, FLEX conta na sua),
```

```

// spend (soma de preço dos titulares), priceOf{pos:[precos]}, maxPrice, avgPri
// _____
// TÁTICAS v4 – ativação pelas AÇÕES dos seus jogadores na partida.
// Cada tática tem uma FAMÍLIA de ações (a "cara" daquela tática).
// Dois testes:
// 1) ESTILO DOMINANTE: a família da tática é a maior fatia das ações
//    ofensivas/defensivas do seu time (proporção interna – não compara com
//    o adversário nem com a média do jogo).
// 2) PARTICIPAÇÃO: pelo menos `minPlayers` dos seus jogadores que entraram
//    contribuíram de verdade naquela família.
// Passou nos 2 → COMPLETA → +12% nas ações da família.
// Falhou em 1 → INCOMPLETA → -5% nas ações da família (ônus < bônus).
// `fam` = chaves de comp (pontuação) que recebem o efeito.
// `metric(p)` = quanto ESTE jogador produziu na família (pra dominância e partic
// `partMin` = mínimo de produção individual pra contar como "participou".
// Alvos de PONTOS no time todo (normalização): toda tática vale o mesmo, indepen
// da família. Completa soma ~+2.5 pts distribuídos entre os jogadores; incomplet
// ~-1.0 pt (ônus sempre menor que o bônus). A distribuição é proporcional ao qua
// cada jogador produziu na família, então quem "fez a tática acontecer" leva mai
const TACT_BONUS_PTS = 3.5;
const TACT_ONUS_PTS = -1.4;
// soma de PONTOS típica de cada família num time (medida nos jogos reais) – usad
// como divisor pra igualar a escala entre táticas de famílias grandes e pequenas
const TACT_PTSREF = { muralha:1.21, pressaototal:1.95, cerebro:6.47, tridente:12.
const TACTICS = {
  muralha:{name:"Estacionar o Ônibus",
    desc:"Defesa raçuda no próprio campo. Ativa se travar o jogo (bloqueios de jo
    fam:["block","clearance","blockShot"], minPlayers:2,
    metric:p=>p.block*2+p.clearance+p.blockShot*4, partMin:1},
  pressaototal:{name:"Gegenpress",
    desc:"Pressão alta pra roubar a bola. Ativa se recuperar a posse (recuperaçõe
    fam:["recovery","tackle","intercept"], minPlayers:2,
    metric:p=>p.recovery+splitTk1(p).tackle*1.5+splitTk1(p).intercept*1.5, partMi
  cerebro:{name:"Tiki-Taka",
    desc:"Criação e troca de passes. Ativa se gerar chances (criação de finalizaç
    fam:["prgp","sca","gca","assist"], minPlayers:2,
    metric:p=>p.prgp*0.15+p.sca*2+p.gca*2+(p.assists?p.assists.length:0)*2, partMi
  tridente:{name:"Ataque Total",
    desc:"Pressão ofensiva total. Ativa se finalizar (gols, finalizações no gol e
    fam:["goal","sot","wasFouled"], minPlayers:2,
    metric:p=>(p.goals?p.goals.length:0)*3+(p.sots?p.sots.length:0)*2+p.wasFouled
  aereo:{name:"Jogo Aéreo",
    desc:"Bola alta e jogo direto. Ativa se o jogo pelo alto (duelos aéreos, cruz
    fam:["aerial","accCross","longBall"], minPlayers:2,
    metric:p=>p.aerial*2+p.accCross*3+p.longBall, partMin:1},
  contra:{name:"Contra-Ataque",
    desc:"Transição rápida conduzindo a bola. Ativa se carregar pra frente (condu

```

```

    fam:["prgCarry","dribbles","pib"], minPlayers:2,
    metric:p=>p.prgCarry*2+p.dribbles*2+p.pib, partMin:1},
};
// famílias de referência pra calcular a DOMINÂNCIA (proporção interna do time).
// IMPORTANTE: cada ação tem volume natural muito diferente (passes >> dribbles >>
// cruzamentos). Por isso normalizamos cada família por um divisor de referência,
// para que "dominante" signifique "o time se destacou NAQUILO relativo ao normal
// daquela ação", e não simplesmente a ação de maior volume bruto (passe sempre v
const TACT_NORM={ muralha:39, pressaototal:77, cerebro:186, tridente:6, aereo:20,
// MINI REBOOT do balanceamento de táticas (z-score):
// cada família tem distribuição muito diferente (passe >> finalização). Em vez d
// comparar famílias entre si (o que sempre favorecia as de ação comum), medimos
// ACIMA DA MÉDIA daquela família o time está, em desvios-padrão (z-score). Assim
// tática ativa com frequência parecida - "dominante" = o time se destacou NAQUIL
// Média e desvio medidos em milhares de times reais de 5 jogadores nos jogos apu
const TACT_MEAN={ muralha:13.04, pressaototal:26.56, cerebro:16.86, tridente:7.12
const TACT_SD={ muralha:6.91, pressaototal:9.36, cerebro:8.48, tridente:5.10, aer
// z-score mínimo por tática pra a família contar como "estilo do time".
// Ajustado por tática (não global) pra equilibrar a frequência de ativação entre
// (todas ~16-24% nos 8 jogos limpos). Régua por-tática: menos elegante que um va
// porém empareia a viabilidade estratégica. Revisar quando houver mais jogos.
const TACT_ZTHRESH={ muralha:0.9, pressaototal:0.85, cerebro:0.85, tridente:0.8,
const TACT_ZTHRESH_DEFAULT=0.5; // fallback se alguma tática não estiver no mapa
// REGRA ANTIGA (v1): usada SÓ pelos jogos já apurados antes do reboot (match.tac
// pra não recalculer pontuações que já valeram. Jogos novos usam o z-score acima
// v1 = top-3 famílias dominantes + NORM antigo + participação antiga (tridente/a
const TACT_NORM_V1={ muralha:45, pressaototal:70, cerebro:113, tridente:11, aereo
const TACT_PART_V1={ tridente:{minPlayers:2,partMin:2}, aereo:{minPlayers:2,partM
const TACT_FAMILIES={
    muralha:p=>p.block*2+p.clearance+p.blockShot*4,
    pressaototal:p=>p.recovery+splitTkl(p).tackle*1.5+splitTkl(p).intercept*1.5,
    cerebro:p=>p.prgp*0.15+p.sca*2+p.gca*3+(p.assists?p.assists.length:0)*3,
    tridente:p=>(p.goals?p.goals.length:0)*3+(p.sots?p.sots.length:0)*2+p.wasFouled
    aereo:p=>p.aerial*2+p.accCross*3+p.longBall,
    contra:p=>p.prgCarry*2+p.dribbles*2+p.pib,
};
// FAMÍLIAS CONGELADAS dos jogos v1 (já apurados) - idênticas às antigas, pra não
const TACT_FAMILIES_V1={
    muralha:p=>p.tklint+p.clearance+p.block,
    pressaototal:p=>p.recovery+p.tklint,
    cerebro:p=>p.prgp+p.sca+p.gca*2,
    tridente:p=>(p.sots?p.sots.length:0)+(p.goals?p.goals.length:0)*2,
    aereo:p=>p.aerial+p.accCross,
    contra:p=>p.dribbles+p.pib,
};
// normaliza um player do match.json pra um objeto completo de stats

```

```

function normP(raw){
  return Object.assign({
    min:0, started:false, goals:[], assists:[], sots:[], dribbles:0, prgp:0, pib:
    sca:0, gca:0, tklint:0, block:0, blockShot:0, recovery:0, aerial:0, clearance
    // tackle/intercept: desarmes e intercepções SEPARADOS (tklint continua sen
    // Usados só nas táticas: Gegenpress=tackle (desarme), Estacionar Ônibus=inte
    tackle:0, intercept:0,
    yellow:0, red:null, errGoal:0, errShot:0, penCom:0, accCross:0, inaccCross:0,
    wasFouled:0, longBall:0, prgCarry:0, dispossessed:0, penaltyWon:0,
    // disputa de pênaltis (shootout): psConv=convertidos | psMiss=NÃO convertido
    psConv:0, psMiss:0, psSave:0,
    // dados de finalização (capturados do shotmap): bola parada e chute de fora
    setPieceSot:0, setPieceGoals:0, longSot:0, longGoals:0
  }, raw||{});
}

// Fallback: jogos antigos só têm tklint (soma). Se tackle/intercept não vierem,
// estima ~55% desarme / ~45% intercepção pra as táticas novas funcionarem retr
function splitTkl(p){
  if((p.tackle||0)===0 && (p.intercept||0)===0 && (p.tklint||0)>0){
    return { tackle: p.tklint*0.55, intercept: p.tklint*0.45 };
  }
  return { tackle: p.tackle||0, intercept: p.intercept||0 };
}

function makeEngine(match){
  const B = match.tacticRules==="v1" ? BASE_V1 : BASE; // pesos antigos p/ jogos
  const GOALS_TL = match.goals_tl||[];
  const endMin = match.endMin||96;
  function scoreAt(min){let h=0,a=0;for(const g of GOALS_TL){if(g.m<min){g.t===ma
  function diffAt(min){const[h,a]=scoreAt(min);return Math.abs(h-a);}
  function extendsLead(team,min){const[h,a]=scoreAt(min);const lead=team===match.
  function cleanSheetHalves(team){const g=GOALS_TL.filter(x=>x.t!==team);return{h
  function liveShare(){let live=0;for(let m=0;m<endMin;m++){if(diffAt(m+0.5)<=1)l
  const LIVE=liveShare();
  const ctxDecisive=d=>d<=1?1.06:0.92;
  const ctxDefEvt=d=>d<=1?1.08:0.94;
  const ctxDefAgg=LIVE*1.08+(1-LIVE)*0.94;
  const ctxSmallAgg=LIVE*1.00+(1-LIVE)*0.90;
  // — BASELINE RELATIVO AO JOGO (táticas equilibradas) —
  // Cada tática ativa quando o time do usuário está entre os ~TOP38% daquele jog
  // na métrica da tática. Assim TODAS têm a mesma chance de ativar em QUALQUER j
  // (um threshold fixo favoreceria jogos de muita posse, muito chute, etc).
  const TACT_PCTL = 0.62; // alvo: ~38% dos times montados ativam (equilíbrio ent
  function _metricsOf(p){return {
    tklintClr: p.tklint+p.clearance,
    sotGoals : p.sots.length + p.goals.length*2,
    prgp      : p.prgp,

```

```

    recovery : p.recovery,
    aerial    : p.aerial,
    sot       : p.sots.length,
    dribPib   : p.dribbles + p.prgCarry*0.5,           // contra-ataque: dribles
    setPiece  : (p.setPieceSot||0) + (p.setPieceGoals||0)*2 + p.aerial*0.5, // bol
    longShot  : (p.longSot||0) + (p.longGoals||0)*2,   // meia-lua: chutes de for
};}

function computeMatchBase(){
    const all=[]; const src=match.players||{};
    for(const id of Object.keys(src)){
        const p=normP(src[id]);
        if(p.min===0||p.subbedOff) continue; // só quem terminou em campo
        all.push(_metricsOf(p));
    }
    const keys=["tklintClr","sotGoals","prgP","recovery","aerial","sot","dribPib"]
    const fallback={tklintClr:14,sotGoals:6,prgP:50,recovery:18,aerial:6,sot:4,dr
    if(all.length<6) return fallback;
    const N=4000, SIZE=5, sums={}; keys.forEach(k=>sums[k]=[]);
    for(let i=0;i<N;i++){
        const acc={}; keys.forEach(k=>acc[k]=0);
        for(let j=0;j<SIZE;j++){ const r=all[(Math.random()*all.length)|0]; keys.fo
        keys.forEach(k=>sums[k].push(acc[k]));
    }
    const base={};
    for(const k of keys){ const arr=sums[k].sort((a,b)=>a-b); base[k]=arr[Math.fl
    // piso mínimo: evita que jogos de poucos eventos zerem o limiar (tática ativ
    const PISO={tklintClr:8,sotGoals:3,prgP:30,recovery:12,aerial:4,sot:3,dribPib
    for(const k of keys){ if(base[k]<PISO[k]) base[k]=PISO[k]; }
    return base;
}

const MATCH_BASE = computeMatchBase();
// — DvG COMPLETO (4 fatores) —
// Força ajustada de um time = ELO×0.55 + Forma×0.30 + Mando×0.15 (+Tabela só
// O bônus underdog vem da diferença de FORÇA AJUSTADA, não só do ELO cru.
// Boa forma sobe a força → reduz o bônus de underdog (o time não está tão por
// Forma de cada time vem opcional em match.form[team] (array dos últimos 5):
//   [{res:"W"|"D"|"L", oppElo:NNNN}, ...] — calculado na captura.
function formaIndex(team){
    const f=match.form&&match.form[team];
    if(!f||!f.length) return null; // sem dados de forma
    let soma=0;
    for(const g of f){
        const base=g.res==="W"?1.0:g.res==="D"?0.5:0.0;
        const pesoOpp=Math.max(0.7,Math.min(1.3,(g.oppElo||1700)/1800));
        soma+=base*pesoOpp;
    }
    // média 0..~1.3 → converte p/ "equivalente ELO" relativo (centrado em 0)

```

```

    const media=soma/f.length;          // 0 (péssima) .. ~1.3 (ótima)
    return (media-0.5)*600;              // -300 (frio) .. +480 (em chamadas)
}
function posTabelaAdj(team){
    // só clubes: match.standing[team] = {pos:N, total:M} (posição na liga)
    const s=match.standing&&match.standing[team];
    if(!s||!s.total) return null;
    // 1º lugar = +200, último = -200 (linear)
    const frac=1-(s.pos-1)/Math.max(1,s.total-1); // 1.0 (líder)..0 (lanterna)
    return (frac-0.5)*400;
}
function forcaAjustada(team){
    let elo=team===match.homeCode?match.homeElo:match.awayElo;
    if(elo===null||isNaN(elo))elo=1700; // fallback: elo faltando não pode virar N
    const mando=match.neutral?0:(team===match.homeCode?+40:-40); // casa vale ~+4
    const forma=formaIndex(team);
    const tabela=posTabelaAdj(team);
    // pesos: ELO .55 / forma .30 / mando .15 (clubes c/ tabela: .50/.25/.10/.15
    if(tabela!==null){
        const fAdj=(forma!==null?forma:0);
        return elo*0.50 + fAdj*0.25 + mando*0.10 + tabela*0.15 + elo*(forma!==null?
    }
    if(forma!==null) return elo*0.55 + forma*0.30 + mando*0.15;
    return elo + mando; // sem forma: cai no modo simples (só ELO+mando)
}
function divgMult(team){
    const opp=team===match.homeCode?match.awayCode:match.homeCode;
    const edge=forcaAjustada(opp)-forcaAjustada(team); // quão + forte é o oponente
    let bonus=Math.min(0.14,Math.max(0,edge/1450)); // bônus underdog por força a
    // BÔNUS DE VISITANTE: em jogo com mando real (não-neutro), o time visitante
    // joga sob pressão de jogar fora → recebe o bônus cheio (+14%), SEMPRE ligado
    // independente de ser favorito ou não. Combina com o underdog mas respeita o
    if(!match.neutral && team===match.awayCode){
        bonus=0.14;
    }
    return 1+bonus;
}
function minFactor(p){if(p.started||p.min>=45)return 1.0;return p.min/90;}
function redPenalty(red){if(!red)return 0;const h1=red.m<=50;if(red.doubleYellow

function indices(p){
    if(p.min===0)return null;
    const per90=v=>v/p.min*90;const pct=(v,c)=>Math.max(0,Math.min(100,v/c*100));
    let iui,tw;
    if(p.gk){const g=p.gk;iui=pct(per90(g.opa+g.crossStop*0.7),2.5);tw=pct(per90(
    else{iui=pct(per90(p.sots.length+p.goals.length+p.sca*1.2+p.gca*2+p.prgp*.85+
        tw=pct(per90(p.tklint+p.block+p.recovery*.5+p.aerial*.5+p.clearance*.3+p

```

```

const ev=[...p.goals.map(g=>tierXG(g.xg).b),...p.assists.map(a=>tierXG(a.xag)
let eff=ev.length?ev.reduce((a,b)=>a+b,0)/ev.length/4.2*100:50;
if(p.gk)eff=Math.max(0,eff-p.gk.conceded*8);
const fW=p.pos==="DEF"?3:4,dW=p.pos==="DEF"?4:6;
const sec=Math.max(0,100-(p.errGoal*45+p.penCom*35+(p.red?40:0)+p.yellow*12+p
return{iui,eff,sec,tw};
}
const lab3=(v,lo,mid,hi)=>v>=65?hi:v>=35?mid:lo;

// =====
// CAMADA COLECIONÁVEL (cosmética - não altera pontuação)
// arch = 1 papel principal do jogador na partida
// traits = até 3 selos de momentos especiais
// rarity = raridade da "carta", de Comum a Lendário
// =====
function archetype(p,ix,clutch,total){
  if(!ix)return{arch:"-",traits:["Não entrou em campo"],rarity:"Comum"};
  const G=p.goals.length, A=p.assists.length;
  const golaco = p.goals.some(g=>tierXG(g.xg).t===4); // xg<0.08
  const golDifícil = p.goals.some(g=>tierXG(g.xg).t>=3);
  const defLimpa = p.gk && p.gk.conceded===0 && p.min>=60;
  // CARRASCO: marcou um gol que tirou o time do empate/derrota e o colocou na
  // (desempate pra liderança ou virada). Avalia o placar logo após cada gol do
  const isCarrasco = p.goals.some(g=>{
    const before=scoreAt(g.m); // placar imediatamente antes do gol
    const my=p.team===match.homeCode?0:1, op=my===0?1:0;
    // antes do gol o time estava empatado ou perdendo, e o gol coloca na frent
    return before[my]<=before[op] && (before[my]+1)>before[op];
  });
  let arch="Conector"; const traits=[];

  // ----- ARQUÉTIPO (prioridade do mais raro/marcante ao mais comum) ----
  if(p.gk){
    const bigSave = p.gk.saves.some(s=>tierSV(s.psxg).t===4);
    const manySaves = p.gk.saves.length>=5;
    if(p.gk.penSave>0) arch="GK Pegador de Pênalti";
    else if((manySaves||bigSave)&&p.gk.conceded===0) arch="GK Paredão";
    else if(manySaves||bigSave) arch="GK Muralha";
    else if((p.gk.opa+p.gk.crossStop)>=3&&ix.sec>=60) arch="GK Líbero";
    else arch="GK Seguro";
  }
  else if(G>=3) arch="Matador"; // hat
  else if(G>=2) arch="Artilheiro"; // 2 g
  else if(!p.gk&&total>=28) arch="GOAT"; // pon
  else if(G>=1&&isCarrasco&&G===1&&total>=10) arch="Carrasco"; // gol
  else if(G>=1&&!p.started) arch="Super Sub"; // ent
  else if(G>=1&&p.setPieceGoals>=1) arch="Especialista de Bola Parada"; // gol

```

```

else if(G>=1&&p.longGoals>=1) arch="Canhão"; // gol
else if(clutch>=2) arch="Herói de Clutch";
else if(G>=1&&golaco) arch="Finalizador Frio";
else if(G>=1&&A>=1) arch="Decisivo"; // gol
else if(A>=3) arch="Rei das Assistências"; // 3+
else if(A>=2) arch="Garçon";
// ---- criação (específicos primeiro, limiares altos) ----
else if((p.sca+p.gca*2)>=9&&p.prgp>=30) arch="Cérebro do Time"; // c
else if((p.sca+p.gca*2)>=7&&p.prgp>=18) arch="Maestro Criador";
// ---- ataque ----
else if(p.pos==="ATT"&&p.dribbles>=6) arch="Driblador"; // dri
else if(p.pos==="ATT"&&p.sots.length>=4) arch="Lobo Solitário"; // mui
else if(p.pos==="ATT"&&p.aerial>=5) arch="Pivô de Área";
else if(p.pos==="ATT"&&(p.sca+p.gca*2)>=5) arch="Camisa 10"; // ata
else if(p.pos==="ATT") arch="Homem de Frente"; // ata
// ---- defesa (específicos primeiro) ----
else if(p.pos==="DEF"&&(p.tklint+p.clearance+p.block)>=16) arch="Xerife"; //
else if(p.pos==="DEF"&&(p.aerial+p.block+p.clearance)>=12) arch="Muralha Aérea";
else if(p.pos==="DEF"&&p.dribbles>=4&&ix.iui>=72) arch="Ala Moderno"; // la
else if(p.pos==="DEF"&&p.prgp>=12&&ix.sec>=72) arch="Zagueiro Construtor"; //
else if(p.pos==="DEF"&&ix.iui>=78&&p.prgp>=4) arch="Lateral de Corredor";
// ---- meio-campo (Box-to-Box agora exige contribuição REAL nas duas fases)
else if(p.pos==="MID"&&(p.tklint+p.recovery)>=7&&(p.sca+p.gca*2+p.dribbles+p.
else if(p.pos==="MID"&&(p.sca+p.gca*2)>=5) arch="Camisa 10";
else if(p.pos==="MID"&&p.dribbles>=5) arch="Condutor"; // co
else if((p.tklint+p.recovery)>=10&&ix.tw>=70) arch="Volante";
else if(ix.tw>=72&&p.recovery>=8) arch="Motor";
else if((p.tklint+p.recovery+p.clearance)>=12&&ix.sec>=74&&p.pos!=="ATT") arc
else if(p.accCross>=4) arch="Maestro de Cruzamentos"; // mui
else if(ix.iui>=72&&ix.sec<42) arch="Ponta Caótico";
else if(p.pos==="MID"&&p.prgp>=8&&ix.iui>=55) arch="Articulador"; // di
else if(p.pos==="MID"&&(p.tklint+p.recovery)>=6) arch="Engrenagem"; // me
else if((p.red||p.penCom>=1||p.errGoal>=1)&&G==0) arch="Vilão"; // fe

// ----- TRAITS (selos de momento - até 3) -----
if(G>=3) traits.push("Hat-trick");
else if(G>=2) traits.push("Dobradinha");
if(A>=2) traits.push("Garçon");
if(golaco) traits.push("Carrasco"); // golaço improvável
if(clutch>0) traits.push("Mente Fria");
if(p.gk&&p.gk.penSave>0) traits.push("Pega-Pênalti");
if(defLimpa) traits.push("Cadeado"); // goleiro/zaga sem sofre
if(!p.gk&&p.sots.length>=3) traits.push("Pé Quente");
if((p.sca+p.gca*2)>=5) traits.push("Maestro");
if(!p.gk&&p.dribbles>=5) traits.push("Pernas de Pau Quebradas"); // drible e
if(G>=1&&A>=1) traits.push("Mão na Massa"); // partici
if(p.pos==="DEF"&&G>=1) traits.push("Zagueiro Artilheiro");

```



```

    if(p.recovery>=10) traits.push("Aspirador"); // recuper
    if(p.aerial>=6) traits.push("Dono do Ar");
    if(p.gk&&p.gk.saves.some(s=>s.psxg>0.6)) traits.push("Paredão");
    if(ix.tw>=88&&p.pos!="ATT") traits.push("Monstro Defensivo");
    if(!p.red&&!p.yellow&&p.fouls===0&&p.dribbledPast===0&&ix.sec>=92) traits.pus
    if(p.red||(p.yellow>=1&&p.fouls>=3)) traits.push("Indisciplinado");
    if(total<1&&p.min>=45) traits.push("Fantasma");
    if(!traits.length) traits.push("Regular");

    // ----- RARIDADE (mais difícil chegar ao topo) -----
    let r=0;
    if(p.goals.some(g=>tierXG(g.xg).t===4&&g.m>=85)) r+=5; // golão decisivo n
    if(golaco) r+=3; // golão improvável
    if(golDificil) r+=1;
    if(G>=3) r+=4; else if(G>=2) r+=2; else if(G>=1) r+=1; // gols
    if(A>=2) r+=2; else if(A>=1) r+=1;
    if(p.gk&&p.gk.penSave>0) r+=3;
    if(p.gk&&p.gk.saves.some(s=>tierSV(s.psxg).t===4)) r+=2;
    if(defLimpa) r+=1;
    if(clutch>=2) r+=2; else if(clutch>0) r+=1;
    if(total>=20) r+=3; else if(total>=15) r+=2; else if(total>=10) r+=1;
    if(traits.includes("Monstro Defensivo")) r+=1;
    if(p.red) r=Math.max(0,r-2);
    if(traits.includes("Fantasma")) r=Math.max(0,r-1);
    const rarity = r>=13?"Lendário":r>=10?"Mítico":r>=7?"Épico":r>=4?"Raro":r>=2?

    return{arch,traits:traits.slice(0,3),rarity};
}

const STAT_DEFS=[
  ["Gols",B.goal,p=>p.goals.length],["Assistências",B.assist,p=>p.assists.lengt
  ["Finalizações no gol",B.sot,p=>p.sots.length],["Dribles certos",B.dribble,p=
  ["Construção ofensiva",B.prgp,p=>p.prgp],["Cruzamentos certos",B.accCross,p=>
  ["Lançamentos certos",B.longBall,p=>p.longBall||0],["Conduções progressivas",
  ["Faltas sofridas",B.wasFouled,p=>p.wasFouled||0],["Pênalti sofrido",B.penalt
  ["Cruzamentos errados",B.inaccCross,p=>p.inaccCross],["Desarmes + interceptaç
  ["Bloqueios",B.block,p=>p.block],["Recuperações de bola",B.recovery,p=>p.reco
  ["Duelos aéreos vencidos",B.aerial,p=>p.aerial],["Cortes",B.clearance,p=>p.cl
  ["Defesas",B.save,p=>p.gk?p.gk.saves.length:0],["Defesa de pênalti",B.penSave
  ["Saídas (sweeper)",B.opa,p=>p.gk?p.gk.opa:0],["Cruzamentos cortados",B.cross
  ["Gols sofridos",B.concededGk,p=>p.gk?p.gk.conceded:0],["Cartão amarelo",B.ye
  ["Faltas",B.foul,p=>p.fouls],["Vezes driblado",B.dribbledPast,p=>p.dribbledPa
  ["Erro → gol",B.errGoal,p=>p.errGoal],["Erro → finalização",B.errShot,p=>p.er
  ["Gol contra",B.ownGoal,p=>p.ownGoal||0],
];

function statLines(p){const o=[];for(const[l,v,c]of STAT_DEFS){const n=c(p);if(

```

```

function scorePlayer(p, tacticKey, squadSum) {
  p=normP(p);
  if (p.min===0) return {total:0, minutes:0, statLines:[], lines:[], evNote:[], labels:
  const ts=match.team_stats?match.team_stats[p.team]:null;
  const lines=[]; const push=(k,v,n)=>{if(Math.abs(v)>=0.05) lines.push([k+(n?` $
  const mf=minFactor(p);
  const comp={
    goal:p.goals.length*B.goal, assist:p.assists.length*B.assist, sotPts:p.sots.l
    dribbles:r1(p.dribbles*mf)*B.dribble, prgp:r1(p.prgp*mf)*B.prgp,
    sca:r1(p.sca*mf)*B.sca, gca:p.gca*B.gca, tklint:r1(p.tklint*mf)*B.tklint, bloc
    recovery:r1(p.recovery*mf)*B.recovery, aerial:r1(p.aerial*mf)*B.aerial, clear
    accCross:r1(p.accCross*mf)*B.accCross, inaccCross:r1(p.inaccCross*mf)*B.inac
    wasFouled:r1(p.wasFouled*mf)*B.wasFouled, longBall:r1(p.longBall*mf)*B.longB
    penaltyWon:(p.penaltyWon||0)*B.penaltyWon,
  };
  let cs=0, csHalves=0; const csEl=p.gk||(p.pos==="DEF"&&p.min>=60);
  if(csEl){const c=cleanSheetHalves(p.team);const gkCS=(match.tacticRules==="v1
  let gkB=0, conc=0;
  if(p.gk){gkB=p.gk.saves.length*B.save+p.gk.opa*B.opa+p.gk.crossStop*B.crossSt
  const negRed=redPenalty(p.red);
  const neg=p.yellow*B.yellow+negRed+p.errGoal*B.errGoal+(p.errShot||0)*B.errSh
  // multiplicador de equilíbrio por posição: só nos POSITIVOS, e só em jogos n
  const posMult=(match.tacticRules==="v1")?1:(match.posMultLegacy?POS_MULT_LEG
  const posPart=(Object.values(comp).reduce((a,b)=>a+b,0)+gkB+cs)*posMult;
  const baseTot=posPart+conc+neg;
  if(mf<1)push(`Stats escalonados (${p.min}', fator ${mf.toFixed(2)})`,0);
  if(cs>0)push(`Sem sofrer gol ${csHalves===2?"(jogo todo)": "(só 1 tempo)"}`,cs
  if(p.red)push(`Vermelho${p.red.doubleYellow?" 2º amarelo":""} ${p.red.m<=50?"
  const sl=statLines(p);
  let dif=0, ctx=0, clutch=0; const ev=[];
  for(const g of p.goals){const t=tierXG(g.xg), d=diffAt(g.m); dif+=t.b; ctx+=(B.g
  for(const a of p.assists){const t=tierXG(a.xag), d=diffAt(a.m); dif+=t.b; ctx+=(
  for(const s of p.sots){const d=diffAt(s.m); ctx+=B.sot*(ctxDecisive(d)-1); if(s
  if(p.gk)for(const s of p.gk.saves){const t=tierSV(s.psxg), d=diffAt(s.m); dif+=
  const defAgg=comp.tklint+comp.block+comp.recovery+comp.aerial+comp.clearance;
  const smallAgg=comp.prgp+comp.sca+comp.dribbles+comp.accCross;
  ctx+=defAgg*(ctxDefAgg-1)+smallAgg*(ctxSmallAgg-1);
  clutch=Math.min(clutch, CAPS.CLUTCH);
  push("Dificuldade (xG·xAG·PSxG)", r1(dif));
  push(LIVE>=0.99?"Placar - jogo vivo o tempo todo":"Contexto de placar", r1(ctx
  if(clutch>0)push(`Clutch fim de jogo/prorrogação (cap +${CAPS.CLUTCH})`, r1(cl
  const posSub=Math.max(0, baseTot-conc-neg)+dif+Math.max(0, ctx)+clutch;
  const dm=dvgMult(p.team); const dvg=posSub*(dm-1);
  if(dvg>0.05){
    const isVisit=!match.neutral && p.team===match.awayCode;
    push(`${isVisit?"Bônus visitante":"DvG underdog"} ×${dm.toFixed(3)}`, r1(dvg
  }
}

```

```

let tact=0;const T=TACTICS[tacticKey];
// TÁTICA – assimetria proposital entre ACERTAR e ERRAR:
// • COMPLETA (full): cada jogador ganha um bônus PROPORCIONAL à própria
//   contribuição na família. Quem produziu muito ganha bastante.
//   famPts = pontos do jogador nas ações da família (T.fam, em comp)
//   ref     = TACT_PTSREF (produção de referência de quem "rende bem")
//   bônus   = TACT_BONUS_PTS × (famPts/ref) – total do time pode passar
// • INCOMPLETA (fail): punição COLETIVA e igual. O time todo escolheu a tática
//   errada, então o ônus (TACT_ONUS_PTS) é dividido em partes iguais entre
//   5 titulares (-0.28 cada). Assim a punição SEMPRE acontece quando se erra
//   tática (no modelo proporcional ela sumia: quem errava mais era punido mais)
if(T&&squadSum&&squadSum.status){
  const st=squadSum.status[tacticKey];
  if(st==="full"){
    let famPts=0; for(const k of T.fam){famPts+=(comp[k]?0);}
    const ref=TACT_PTSREF[tacticKey]||10;
    tact=TACT_BONUS_PTS*(famPts/ref); // proporcional à contribuição
  }else{
    tact=TACT_ONUS_PTS/5; // ônus coletivo: parte igual do alvo entre os 5 titulares
  }
  // teto do bônus tático: jogos novos (match.tactCapV2===true) usam 4
  // (tática = tempero, não motor); jogos já apurados antes dessa mudança
  // mantêm o teto antigo (CAPS.TACT=13) pra não alterar resultados publicados
  const tactCap = match.tactCapV2 ? 4 : CAPS.TACT;
  tact=Math.max(-tactCap,Math.min(tactCap,tact));
  if(Math.abs(tact)>=0.05)push(`Tática ${T.name} ${st==="full"?"completa":"incompleta"}
  `);
  const ix=indices(p);const avg=ix.iui*.3+ix.eff*.3+ix.sec*.2+ix.tw*.2;const pe=
  push("Performance (índices C+)",perf);
  let total=baseTot+dif+ctx+clutch+dvgt+tact+perf;
  const cap=Math.max(CAPS.FLOOR,Math.min(CAPS.MATCH,total));
  if(cap!==total)push(`Cap (${CAPS.FLOOR}/${CAPS.MATCH})`,r1(cap-total));
  total=r1(cap);
  // disputa de pênaltis (shootout): evento extra decisivo, somado FORA do cap
  let shoot=0;
  if(p.psConv>0){shoot+=p.psConv*B.psConv;push(`Pênalti convertido (disputa) ${p.psConv}`);
  if(p.psSave>0){shoot+=p.psSave*B.psSave;push(`Pênalti defendido (disputa) ${p.psSave}`);
  if(p.psMiss>0){shoot+=p.psMiss*B.psMiss;push(`Pênalti perdido (disputa) ${p.psMiss}`);
  if(shoot!==0)total=r1(total+shoot);
  const meta=archetype(p,ix,clutch,total);
  const labels=[`Envolvimento: ${lab3(ix.iui,"discreto","participativo","onipre")}`];
  return{total,minutes:p.min,statLines:sl,lines:lines.map(([k,v])=>[k,r1(v)]),e
}

// Avalia a TÁTICA pelas ações dos jogadores que ENTRARAM (min>0), incluindo
// os que foram substituídos. Retorna, para cada tática, o status:
// 'full' → estilo dominante E participação atingida → bônus

```

```

// 'fail' → faltou um dos dois → ônus
// (a tática escolhida pelo usuário é lida deste mapa no scorePlayer)
function squadSum(players){
  const ps=[];
  for(const raw of players){const p=normP(raw);if(p.min>0)ps.push(p);}
  const useV1 = match.tacticRules==="v1"; // jogos já apurados antes do reboot
  // soma bruta de cada família no time
  const famRaw={},famNorm={},famZ={};
  const FAMSET = useV1?TACT_FAMILIES_V1:TACT_FAMILIES;
  for(const k of Object.keys(FAMSET)){famRaw[k]=0;}
  for(const p of ps){for(const k of Object.keys(FAMSET))famRaw[k]+=FAMSET[k](p)}
  const NORMSET = useV1?TACT_NORM_V1:TACT_NORM;
  for(const k of Object.keys(FAMSET)){
    famNorm[k]=famRaw[k]/(NORMSET[k]||1);
    famZ[k]=(famRaw[k]-(TACT_MEAN[k]||0))/(TACT_SD[k]||1);
  }
  // v1: dominante = estar entre as 3 famílias mais fortes (normalizadas)
  let topSet=null;
  if(useV1){
    const ranked=Object.entries(famNorm).sort((a,b)=>b[1]-a[1]).map(e=>e[0]);
    topSet=new Set(ranked.slice(0,3));
  }
  const status={};
  for(const[key,T]of Object.entries(TACTICS)){
    let dominant, minP=T.minPlayers, partMin=T.partMin;
    if(useV1){
      // regra antiga: top-3 + participação antiga (tridente/aereo pediam mais
      dominant = (famNorm[key]>0) && topSet.has(key);
      const pv=TACT_PART_V1[key]; if(pv){minP=pv.minPlayers;partMin=pv.partMin;}
    }else{
      // regra nova: z-score acima do limiar (régua justa entre todas)
      dominant = (famZ[key]||0)>=(TACT_ZTHRESH[key]!==null?TACT_ZTHRESH[key]:TAC
    }
    let part=0; for(const p of ps){ if(T.metric(p)>=partMin) part++; }
    const enough = part>=minP;
    status[key] = (dominant&&enough) ? "full" : "fail";
  }
  return {status, famSum:famRaw, famNorm, famZ, n:ps.length};
}

return { scorePlayer, squadSum, TACTICS, matchBase:MATCH_BASE };
}

if (typeof module!=="undefined" && module.exports) module.exports={makeEngine,TAC
if (typeof window!=="undefined"){ window.makeEngine=makeEngine; window.ENGINE_TAC

```

